ASSIGNMENT-12.5

H.no-2303A51L86

Batch – 30

Task 1 – Sorting – Merge Sort Implementation

Prompt :

#generate a Python program that implements the Merge Sort algorithm.create a function merge_sort(arr) that sorts a list in ascending order.include time complexity and space complexity in the function docstring.

CODE:

```python
#generate a Python program that implements the Merge Sort algorithm.create a function merge_sort(arr) that sorts a list in ascending order.include time complexity and space complexity in the functio
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.

    Time Complexity: O(n log n) - where n is the number of elements in the list.
    Space Complexity: O(n) - due to the temporary arrays used for merging.

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: A new sorted list in ascending order.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    """Merges two sorted lists into a single sorted list.
    Parameters:    left (list): The first sorted list.
    right (list): The second sorted list.
    Returns:    list: A merged sorted list containing all elements from both input lists.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in the left half, add them to merged
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    # If there are remaining elements in the right half, add them to merged
    while right_index < len(right):
        merged.append(right[right_index])
```

```python
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in the left half, add them to merged
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    # If there are remaining elements in the right half, add them to merged
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
# Example usage:
if __name__ == "__main__":
    unsorted_list = [38, 27, 43, 3, 9, 82, 10]
    sorted_list = merge_sort(unsorted_list)
    print("Sorted list:", sorted_list)
```

OUTPUT:

```
Sorted list: [3, 9, 10, 27, 38, 43, 82]
```

JUSTIFICATION:

Merge sort is used because it divides the list into smaller parts and then merges them after sorting. It works efficiently for large data. The time complexity is O(n log n) which makes it faster than simple sorting methods like bubble sort.

Task 2 – Searching – Binary Search with AI Optimization

Prompt :

#create a binary search function that finds a target element in a sorted list.create a function binary_search(arr,target) returning the index of the target or -1 if not found.

#Include docstrings explaining best, average, and worst-case complexities.

CODE:

```python
#create a binary search function that finds a target element in a sorted list.create a function binary_search(arr,target) returning the index of the target or -1 if not found.
#Include docstrings explaining best, average, and worst-case complexities.
def binary_search(arr, target):
    """
    Performs binary search to find the index of the target element in a sorted list.

    Best Case Complexity: O(1) - when the target is at the middle of the list.
    Average Case Complexity: O(log n) - where n is the number of elements in the list.
    Worst Case Complexity: O(log n) - when the target is not found or is at the end of the list.

    Parameters:
    arr (list): The sorted list to search through.
    target: The element to find in the list.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the list
    return -1
# Example usage:
if __name__ == "__main__":
    sorted_list = [3, 9, 10, 27, 38, 43, 82]
    target = 10
    index = binary_search(sorted_list, target)
    if index != -1:
        print(f"Target {target} found at index: {index}")
    else:
        print(f"Target {target} not found in the list.")
```

OUTPUT:

```
Target 10 found at index: 2
```

JUSTIFICATION:

Binary search is used to find an element in a sorted list. It works by dividing the list into halves and checking the middle element. This makes searching faster with O(log n) time complexity.

Task 3 – Smart Healthcare Appointment Scheduling System

Prompt :

#A healthcare platform maintains appointment records containing

appointment ID, patient name, doctor name, appointment time, and

consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

recommend suitable searching and sorting algorithms.

Justify the selected algorithms.

Implement the algorithms in Python.

CODE:

```python
#A healthcare platform maintains appointment records containing
# appointment ID, patient name, doctor name, appointment time, and
# consultation fee. The system needs to:
# 1. Search appointments using appointment ID.
# 2. Sort appointments based on time or consultation fee.
# recommend suitable searching and sorting
# algorithms.
# Justify the selected algorithms.
# Implement the algorithms in Python.
'''For searching appointments using appointment ID, a su"itable algorithm would be Binary Search, provided that the appointment records are sorted by appointment ID. Binary Search is efficient with
For sorting appointments based on time or consultation fee, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of O(n log n) and is stable, meaning it preserves the rela
Here is the implementation of both algorithms in Python:'''
class Appointment:
    def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee
def binary_search_appointments(appointments, target_id):
    """
    Searches for an appointment by appointment ID using Binary Search.

    Time Complexity: O(log n) - where n is the number of appointments.
    Space Complexity: O(1) - as it uses constant extra space.

    Parameters:
    appointments (list): A list of Appointment objects sorted by appointment_id.
    target_id: The appointment ID to search for.

    Returns:
    Appointment: The appointment object if found, otherwise None.
    """
    left, right = 0, len(appointments) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if appointments[mid].appointment_id == target_id:
            return appointments[mid]
        elif appointments[mid].appointment_id < target_id:
            left = mid + 1
        else:
            right = mid - 1

    return None
def merge_sort_appointments(appointments, key):
```

```python
    Sorts a list of Appointment objects based on a specified key using Merge Sort.
    Time Complexity: O(n log n) - where n is the number of appointments.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    Parameters:
    appointments (list): A list of Appointment objects to be sorted.
    key (str): The attribute to sort by ('appointment_time' or 'consultation_fee').
    Returns:
    list: A new sorted list of Appointment objects based on the specified key.
    """
    if len(appointments) <= 1:
        return appointments

    mid = len(appointments) // 2
    left_half = merge_sort_appointments(appointments[:mid], key)
    right_half = merge_sort_appointments(appointments[mid:], key)

    return merge_appointments(left_half, right_half, key)
def merge_appointments(left, right, key):
    """Merges two sorted lists of Appointment objects based on a specified key.
    Parameters:    left (list): The first sorted list of Appointment objects.
    right (list): The second sorted list of Appointment objects.
    key (str): The attribute to sort by ('appointment_time' or 'consultation_fee').
    Returns:    list: A merged sorted list of Appointment objects based on the specified key.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if getattr(left[left_index], key) < getattr(right[right_index], key):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
```

```python
    return merged
# Example usage:
if __name__ == "__main__":
    appointments = [
        Appointment(1, "Alice", "Dr. Smith", "2024-07-01 10:00", 100),
        Appointment(2, "Bob", "Dr. Jones", "2024-07-01 11:00", 150),
        Appointment(3, "Charlie", "Dr. Brown", "2024-07-01 09:00", 120),
    ]
    # Sort appointments by time
    sorted_by_time = merge_sort_appointments(appointments, key='appointment_time')
    print("Appointments sorted by time:")
    for appt in sorted_by_time:
        print(f"{appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} for ${appt.consultation_fee}")

    # Search for an appointment by ID
    target_id = 2
    found_appointment = binary_search_appointments(sorted_by_time, target_id)
    if found_appointment:
        print(f"\nAppointment found: {found_appointment.patient_name} with {found_appointment.doctor_name} at {found_appointment.appointment_time} for ${found_appointment.consultation_fee}")
    else:
        print(f"\nAppointment with ID {target_id} not found.")
```

OUTPUT:

```
Appointments sorted by time:
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Bob with Dr. Jones at 2024-07-01 11:00 for $150

Appointment found: Bob with Dr. Jones at 2024-07-01 11:00 for $150
```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search is used to find the appointment using appointment ID because IDs are unique and can be sorted. It helps in finding the appointment quickly.

Sorting Algorithm: Merge Sort

Merge sort is used to sort appointments based on time or consultation fee. It works efficiently for large records and gives consistent performance.

Task 4 – Railway Ticket Reservation System Scenario

Prompt :

#A railway reservation system stores booking details such as ticket

ID, passenger name, train number, seat number, and travel date. The

system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Identify efficient algorithms using AI assistance.

• Justify the algorithm choices.

• Implement searching and sorting in Python

CODE:

```python
#A railway reservation system stores booking details such as ticket
# ID, passenger name, train number, seat number, and travel date. The
# system must:
# 1. Search tickets using ticket ID.
# 2. Sort bookings based on travel date or seat number.
# Identify efficient algorithms using AI assistance.
# • Justify the algorithm choices.
# • Implement searching and sorting in Python.
'''For searching tickets using ticket ID, a suitable algorithm would be Binary Search, provided that the booking records are sorted by ticket ID. Binary Search is efficient with a time complexity o
For sorting bookings based on travel date or seat number, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of O(n log n) and is stable, meaning it preserves the relat
Here is the implementation of both algorithms in Python:'''
class Booking:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date
def binary_search_bookings(bookings, target_id):
    """
    Searches for a booking by ticket ID using Binary Search.

    Time Complexity: O(log n) - where n is the number of bookings.
    Space Complexity: O(1) - as it uses constant extra space.

    Parameters:
    bookings (list): A list of Booking objects sorted by ticket_id.
    target_id: The ticket ID to search for.

    Returns:
    Booking: The booking object if found, otherwise None.
    """
    left, right = 0, len(bookings) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if bookings[mid].ticket_id == target_id:
            return bookings[mid]
        elif bookings[mid].ticket_id < target_id:
            left = mid + 1
        else:
            right = mid - 1

    return None
def merge_sort_bookings(bookings, key):
```

```python
def merge_sort_bookings(bookings, key):
    """Sorts a list of Booking objects based on a specified key using Merge Sort.
    Time Complexity: O(n log n) - where n is the number of bookings.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    Parameters:
    bookings (list): A list of Booking objects to be sorted.
    key (str): The attribute to sort by ('travel_date' or 'seat_number').
    Returns:
    list: A new sorted list of Booking objects based on the specified key.
    """
    if len(bookings) <= 1:
        return bookings

    mid = len(bookings) // 2
    left_half = merge_sort_bookings(bookings[:mid], key)
    right_half = merge_sort_bookings(bookings[mid:], key)

    return merge_bookings(left_half, right_half, key)
def merge_bookings(left, right, key):
    """Merges two sorted lists of Booking objects based on a specified key.
    Parameters:
    left (list): The first sorted list of Booking objects.
    right (list): The second sorted list of Booking objects.
    key (str): The attribute to sort by ('travel_date' or 'seat_number').
    Returns:
    list: A merged sorted list of Booking objects based on the specified key.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if getattr(left[left_index], key) < getattr(right[right_index], key):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
```

```python
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
# Example usage:
if __name__ == "__main__":
    bookings = [
        Booking(1, "Alice", "Train A", "12A", "2024-08-01"),
        Booking(2, "Bob", "Train B", "15B", "2024-08-02"),
        Booking(3, "Charlie", "Train A", "10C", "2024-08-01"),
    ]
    # Sort bookings by travel date
    sorted_by_date = merge_sort_bookings(bookings, key='travel_date')
    print("Bookings sorted by travel date:")
    for booking in sorted_by_date:
        print(f"{booking.passenger_name} on {booking.train_number} seat {booking.seat_number} for {booking.travel_date}")

    # Search for a booking by ticket ID
    target_id = 2
    found_booking = binary_search_bookings(sorted_by_date, target_id)
    if found_booking:
        print(f"\nBooking found: {found_booking.passenger_name} on {found_booking.train_number} seat {found_booking.seat_number} for {found_booking.travel_date}")
    else:
        print(f"\nBooking with ID {target_id} not found.")
```

OUTPUT:

```
Bookings sorted by travel date:
Charlie on Train A seat 10C for 2024-08-01
Alice on Train A seat 12A for 2024-08-01
Bob on Train B seat 15B for 2024-08-02

Booking found: Bob on Train B seat 15B for 2024-08-02
```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search helps to find ticket details using ticket ID quickly. It reduces the number of comparisons compared to linear search.

Sorting Algorithm: Merge Sort

Merge sort is used to sort booking details based on travel date or seat number. It works efficiently even when the number of bookings increases.

Task 5 – Smart Hostel Room Allocation System

Prompt :

#Smart Hostel Room Allocation System

A hostel management system storesstudent room allocation details

including student ID, room number, floor, and allocation date. The

system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

suggest optimized algorithms.

• Justify the selections.

• Implement the solution in Python.

CODE:

```python
#Smart Hostel Room Allocation System
# A hostel management system stores student room allocation details
# including student ID, room number, floor, and allocation date. The
# system needs to:
# 1. Search allocation details using student ID.
# 2. Sort records based on room number or allocation date.
# suggest optimized algorithms.
# • Justify the selections.
# • Implement the solution in Python.
'''For searching allocation details using student ID, a suitable algorithm would be Binary Search, provided that the allocation records are sorted by student ID. Binary Search is efficient with a t
For sorting records based on room number or allocation date, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of O(n log n) and is stable, meaning it preserves the re
Here is the implementation of both algorithms in Python:'''
class Allocation:
    def __init__(self, student_id, room_number, floor, allocation_date):
        self.student_id = student_id
        self.room_number = room_number
        self.floor = floor
        self.allocation_date = allocation_date
def binary_search_allocations(allocations, target_id):
    """
    Searches for an allocation by student ID using Binary Search.

    Time Complexity: O(log n) - where n is the number of allocations.
    Space Complexity: O(1) - as it uses constant extra space.

    Parameters:
    allocations (list): A list of Allocation objects sorted by student_id.
    target_id: The student ID to search for.

    Returns:
    Allocation: The allocation object if found, otherwise None.
    """
    left, right = 0, len(allocations) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if allocations[mid].student_id == target_id:
            return allocations[mid]
        elif allocations[mid].student_id < target_id:
            left = mid + 1
        else:
            right = mid - 1

    return None
def merge_sort_allocations(allocations, key):
    """Sorts a list of Allocation objects based on a specified key using Merge Sort.
```

```python
def merge_sort_allocations(allocations, key):
    """Sorts a list of Allocation objects based on a specified key using Merge Sort.
    Time Complexity: O(n log n) - where n is the number of allocations.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    Parameters:
    allocations (list): A list of Allocation objects to be sorted.
    key (str): The attribute to sort by ('room_number' or 'allocation_date').
    Returns:
    list: A new sorted list of Allocation objects based on the specified key.
    """
    if len(allocations) <= 1:
        return allocations

    mid = len(allocations) // 2
    left_half = merge_sort_allocations(allocations[:mid], key)
    right_half = merge_sort_allocations(allocations[mid:], key)

    return merge_allocations(left_half, right_half, key)
def merge_allocations(left, right, key):
    """Merges two sorted lists of Allocation objects based on a specified key.
    Parameters:
    left (list): The first sorted list of Allocation objects.
    right (list): The second sorted list of Allocation objects.
    key (str): The attribute to sort by ('room_number' or 'allocation_date').
    Returns:
    list: A merged sorted list of Allocation objects based on the specified key.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if getattr(left[left_index], key) < getattr(right[right_index], key):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
```

```python
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
# Example usage:
if __name__ == "__main__":
    allocations = [
        Allocation(1, "101", "1st Floor", "2024-09-01"),
        Allocation(2, "102", "1st Floor", "2024-09-02"),
        Allocation(3, "201", "2nd Floor", "2024-09-01"),
    ]
    # Sort allocations by room number
    sorted_by_room = merge_sort_allocations(allocations, key='room_number')
    print("Allocations sorted by room number:")
    for alloc in sorted_by_room:
        print(f"Student ID: {alloc.student_id}, Room: {alloc.room_number}, Floor: {alloc.floor}, Date: {alloc.allocation_date}")

    # Search for an allocation by student ID
    target_id = 2
    found_allocation = binary_search_allocations(sorted_by_room, target_id)
    if found_allocation:
        print(f"\nAllocation found: Student ID: {found_allocation.student_id}, Room: {found_allocation.room_number}, Floor: {found_allocation.floor}, Date: {found_allocation.allocation_date}")
    else:
        print(f"\nAllocation with Student ID {target_id} not found.")
```

OUTPUT:

```
Allocations sorted by room number:
Student ID: 1, Room: 101, Floor: 1st Floor, Date: 2024-09-01
Student ID: 2, Room: 102, Floor: 1st Floor, Date: 2024-09-02
Student ID: 3, Room: 201, Floor: 2nd Floor, Date: 2024-09-01

Allocation found: Student ID: 2, Room: 102, Floor: 1st Floor, Date: 2024-09-02
```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search is used to search student room allocation using student ID. Since student IDs are unique,the search becomes faster.

Sorting Algorithm: Merge Sort

Merge sort is used to sort records by room number or allocation date. It provides efficient sorting for large data.

Task 6 – Online Movie Streaming Platform

Prompt :

#Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Recommend searching and sorting algorithms .

• Justify the chosen algorithms.

• Implement Python functions.

CODE:

```python
#Online Movie Streaming Platform
# A streaming service maintains movie records with movie ID, title,
# genre, rating, and release year. The platform needs to:
# 1. Search movies by movie ID.
# 2. Sort movies based on rating or release year.
# Recommend searching and sorting algorithms .
# • Justify the chosen algorithms.
# • Implement Python functions.
'''For searching movies by movie ID, a suitable algorithm would be Binary Search, provided that the movie records are sorted by movie ID. Binary Search is efficient with a time complexity of O(log
For sorting movies based on rating or release year, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of O(n log n) and is stable, meaning it preserves the relative or
Here is the implementation of both algorithms in Python:'''
class Movie:
    def __init__(self, movie_id, title, genre, rating, release_year):
        self.movie_id = movie_id
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year
def binary_search_movies(movies, target_id):
    """
    Searches for a movie by movie ID using Binary Search.

    Time Complexity: O(log n) - where n is the number of movies.
    Space Complexity: O(1) - as it uses constant extra space.

    Parameters:
    movies (list): A list of Movie objects sorted by movie_id.
    target_id: The movie ID to search for.

    Returns:
    Movie: The movie object if found, otherwise None.
    """
    left, right = 0, len(movies) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if movies[mid].movie_id == target_id:
            return movies[mid]
        elif movies[mid].movie_id < target_id:
            left = mid + 1
        else:
            right = mid - 1

    return None
def merge_sort_movies(movies, key):
    """Sorts a list of Movie objects based on a specified key using Merge Sort.
```

```python
def merge_sort_movies(movies, key):
    """Sorts a list of Movie objects based on a specified key using Merge Sort.
    Time Complexity: O(n log n) - where n is the number of movies.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    Parameters:
    movies (list): A list of Movie objects to be sorted.
    key (str): The attribute to sort by ('rating' or 'release_year').
    Returns:
    list: A new sorted list of Movie objects based on the specified key.
    """
    if len(movies) <= 1:
        return movies

    mid = len(movies) // 2
    left_half = merge_sort_movies(movies[:mid], key)
    right_half = merge_sort_movies(movies[mid:], key)

    return merge_movies(left_half, right_half, key)
def merge_movies(left, right, key):
    """Merges two sorted lists of Movie objects based on a specified key.
    Parameters:
    left (list): The first sorted list of Movie objects.
    right (list): The second sorted list of Movie objects.
    key (str): The attribute to sort by ('rating' or 'release_year').
    Returns:
    list: A merged sorted list of Movie objects based on the specified key.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if getattr(left[left_index], key) < getattr(right[right_index], key):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
```

```python
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
# Example usage:
if __name__ == "__main__":
    movies = [
        Movie(1, "Movie A", "Action", 8.5, 2020),
        Movie(2, "Movie B", "Comedy", 7.2, 2019),
        Movie(3, "Movie C", "Drama", 9.0, 2021),
    ]
    # Sort movies by rating
    sorted_by_rating = merge_sort_movies(movies, key='rating')
    print("Movies sorted by rating:")
    for movie in sorted_by_rating:
        print(f"{movie.title} ({movie.genre}) - Rating: {movie.rating}, Release Year: {movie.release_year}")

    # Search for a movie by ID
    target_id = 2
    found_movie = binary_search_movies(sorted_by_rating, target_id)
    if found_movie:
        print(f"\nMovie found: {found_movie.title} ({found_movie.genre}) - Rating: {found_movie.rating}, Release Year: {found_movie.release_year}")
    else:
        print(f"\nMovie with ID {target_id} not found.")
```

OUTPUT:

```
Movies sorted by rating:
Movie B (Comedy) - Rating: 7.2, Release Year: 2019
Movie A (Action) - Rating: 8.5, Release Year: 2020
Movie C (Drama) - Rating: 9.0, Release Year: 2021

Movie with ID 2 not found.
```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search helps to quickly find movies using movie ID in the system.

Sorting Algorithm: Merge Sort

Merge sort is used to sort movies based on rating or release year because it works efficiently for large movie databases.

Task 7 – Smart Agriculture Crop Monitoring System

Prompt :

#Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop

name, soil moisture level, temperature, and yield estimate. Farmers

need to:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate.

select algorithms.

• Justify algorithm suitability.

• Implement searching and sorting in Python.

CODE:

```python
#Smart Agriculture Crop Monitoring System
# An agriculture monitoring system stores crop data with crop ID, crop
# name, soil moisture level, temperature, and yield estimate. Farmers
# need to:
# 1. Search crop details using crop ID.
# 2. Sort crops based on moisture level or yield estimate.
# select algorithms.
# • Justify algorithm suitability.
# • Implement searching and sorting in Python.
'''For searching crop details using crop ID, a suitable algorithm would be Binary Search, provided that the crop records are sorted by crop ID. Binary Search is efficient with a time complexity of O
For sorting crops based on moisture level or yield estimate, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of O(n log n) and is stable, meaning it preserves the rel
Here is the implementation of both algorithms in Python:'''
class Crop:
    def __init__(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):
        self.crop_id = crop_id
        self.crop_name = crop_name
        self.soil_moisture_level = soil_moisture_level
        self.temperature = temperature
        self.yield_estimate = yield_estimate
def binary_search_crops(crops, target_id):
    """
    Searches for a crop by crop ID using Binary Search.

    Time Complexity: O(log n) - where n is the number of crops.
    Space Complexity: O(1) - as it uses constant extra space.

    Parameters:
    crops (list): A list of Crop objects sorted by crop_id.
    target_id: The crop ID to search for.

    Returns:
    Crop: The crop object if found, otherwise None.
    """
    left, right = 0, len(crops) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if crops[mid].crop_id == target_id:
            return crops[mid]
        elif crops[mid].crop_id < target_id:
            left = mid + 1
        else:
            right = mid - 1

    return None
def merge_sort_crops(crops, key):
```

```python
    """Sorts a list of Crop objects based on a specified key using Merge Sort.
    Time Complexity: O(n log n) - where n is the number of crops.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    Parameters:
    crops (list): A list of Crop objects to be sorted.
    key (str): The attribute to sort by ('soil_moisture_level' or 'yield_estimate').
    Returns:
    list: A new sorted list of Crop objects based on the specified key.
    """
    if len(crops) <= 1:
        return crops

    mid = len(crops) // 2
    left_half = merge_sort_crops(crops[:mid], key)
    right_half = merge_sort_crops(crops[mid:], key)

    return merge_crops(left_half, right_half, key)
def merge_crops(left, right, key):
    """Merges two sorted lists of Crop objects based on a specified key.
    Parameters:
    left (list): The first sorted list of Crop objects.
    right (list): The second sorted list of Crop objects.
    key (str): The attribute to sort by ('soil_moisture_level' or 'yield_estimate').
    Returns:
    list: A merged sorted list of Crop objects based on the specified key.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if getattr(left[left_index], key) < getattr(right[right_index], key):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
```

```python
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
# Example usage:
if __name__ == "__main__":
    crops = [
        Crop(1, "Wheat", 30, 25, 1000),
        Crop(2, "Corn", 40, 30, 1500),
        Crop(3, "Rice", 35, 28, 1200),
    ]
    # Sort crops by soil moisture level
    sorted_by_moisture = merge_sort_crops(crops, key='soil_moisture_level')
    print("Crops sorted by soil moisture level:")
    for crop in sorted_by_moisture:
        print(f"{crop.crop_name} - Moisture Level: {crop.soil_moisture_level}, Yield Estimate: {crop.yield_estimate}")

    # Search for a crop by ID
    target_id = 2
    found_crop = binary_search_crops(sorted_by_moisture, target_id)
    if found_crop:
        print(f"\nCrop found: {found_crop.crop_name} - Moisture Level: {found_crop.soil_moisture_level}, Yield Estimate: {found_crop.yield_estimate}")
    else:
        print(f"\nCrop with ID {target_id} not found.")
```

OUTPUT:

```
Crops sorted by soil moisture level:
Wheat - Moisture Level: 30, Yield Estimate: 1000
Rice - Moisture Level: 35, Yield Estimate: 1200
Corn - Moisture Level: 40, Yield Estimate: 1500

Crop with ID 2 not found.
```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search is used to search crop details using crop ID quickly.

Sorting Algorithm: Merge Sort

Merge sort helps to sort crop data based on moisture level or yield estimate in an efficient way.

Task 8 – Airport Flight Management System

Prompt :

#Airport Flight Management System

An airport system stores flight information including flight ID,

airline name, departure time, arrival time, and status. The system

must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

#select other than merge sort

• Justify the algorithm choices.

• Implement the solution in Python.

CODE:

```python
#Airport Flight Management System
# An airport system stores flight information including flight ID,
# airline name, departure time, arrival time, and status. The system
# must:
# 1. Search flight details using flight ID.
# 2. Sort flights based on departure time or arrival time.
#select other than merge sort
# • Justify the algorithm choices.
# • Implement the solution in Python.
'''For searching flight details using flight ID, a suitable algorithm would be Binary Search, provided that
the flight records are sorted by flight ID. Binary Search is efficient with a time complexity of O(log n) and is ideal for large datasets, making it a good choice for searching through flight record
For sorting flights based on departure time or arrival time, the Quick Sort algorithm is a suitable choice. Quick Sort has an average time complexity of O(n log n) and is an in-place sorting algorit
Here is the implementation of both algorithms in Python:'''
class Flight:
    def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
        self.flight_id = flight_id
        self.airline_name = airline_name
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status
def binary_search_flights(flights, target_id):
    """
    Searches for a flight by flight ID using Binary Search.

    Time Complexity: O(log n) - where n is the number of flights.
    Space Complexity: O(1) - as it uses constant extra space.

    Parameters:
    flights (list): A list of Flight objects sorted by flight_id.
    target_id: The flight ID to search for.

    Returns:
    Flight: The flight object if found, otherwise None.
    """
    left, right = 0, len(flights) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if flights[mid].flight_id == target_id:
            return flights[mid]
        elif flights[mid].flight_id < target_id:
            left = mid + 1
        else:
            right = mid - 1

    return None
```

```python
def quick_sort_flights(flights, key):
    """Sorts a list of Flight objects based on a specified key using Quick Sort.
    Time Complexity: O(n log n) on average, O(n^2) in the worst case.
    Space Complexity: O(log n) on average, O(n) in the worst case due to recursion.
    Parameters:
    flights (list): A list of Flight objects to be sorted.
    key (str): The attribute to sort by ('departure_time' or 'arrival_time').
    Returns:
    list: A new sorted list of Flight objects based on the specified key.
    """
    if len(flights) <= 1:
        return flights

    pivot = flights[len(flights) // 2]
    left = [x for x in flights if getattr(x, key) < getattr(pivot, key)]
    middle = [x for x in flights if getattr(x, key) == getattr(pivot, key)]
    right = [x for x in flights if getattr(x, key) > getattr(pivot, key)]

    return quick_sort_flights(left, key) + middle + quick_sort_flights(right, key)
# Example usage:
if __name__ == "__main__":
    flights = [
        Flight(1, "Airline A", "2024-10-01 08:00", "2024-10-01 10:00", "On Time"),
        Flight(2, "Airline B", "2024-10-01 09:00", "2024-10-01 11:00", "Delayed"),
        Flight(3, "Airline C", "2024-10-01 07:00", "2024-10-01 09:00", "On Time"),
    ]
    # Sort flights by departure time
    sorted_by_departure = quick_sort_flights(flights, key='departure_time')
    print("Flights sorted by departure time:")
    for flight in sorted_by_departure:
        print(f"{flight.airline_name} - Departure: {flight.departure_time}, Arrival: {flight.arrival_time}, Status: {flight.status}")

    # Search for a flight by ID
    target_id = 2
    found_flight = binary_search_flights(sorted_by_departure, target_id)
    if found_flight:
        print(f"\nFlight found: {found_flight.airline_name} - Departure: {found_flight.departure_time}, Arrival: {found_flight.arrival_time}, Status: {found_flight.status}")
    else:
        print(f"\nFlight with ID {target_id} not found.")
```

OUTPUT:

```
Flights sorted by departure time:
Airline C - Departure: 2024-10-01 07:00, Arrival: 2024-10-01 09:00, Status: On Time
Airline A - Departure: 2024-10-01 08:00, Arrival: 2024-10-01 10:00, Status: On Time
Airline B - Departure: 2024-10-01 09:00, Arrival: 2024-10-01 11:00, Status: Delayed

Flight found: Airline B - Departure: 2024-10-01 09:00, Arrival: 2024-10-01 11:00, Status: Delayed
```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search is used to find flight details using flight ID quickly.

Sorting Algorithm: Merge Sort

Merge sort is used to sort flights based on departure time or arrival time efficiently.